National University of Computer and Emerging Sciences

AL-2002: Artificial Intelligence
Lab Manual 05

Department of Computer Science
FAST-NU, Lahore, Pakistan

Table of Contents

Objectives 3
1 Informed Search Algorithms.....3
1.1 What is a Heuristic? 3
1.2 Types of Informed Search Algorithms.....4
2 Game Playing in Artificial Intelligence 5
2.1 Characteristics of Game Playing Problems.....5
2.2 Game State Representation 6
2.3 Utility (Evaluation) Function.....6
3 Minimax Algorithm.....6
3.1 Node Representation 8

Objectives

After performing this lab, students shall be able to understand implementation of the following :

- ✔ Informed (heuristic-based) search algorithms:Greedy BFS, A*
- ✔ Game Playing using minimax algorithm

1 Informed Search Algorithms

Informed search algorithms (also called heuristic search algorithms) are search techniques that use additional problem-specific knowledge to guide the search toward the goal more efficiently.

Unlike blind search algorithms, informed search algorithms use a heuristic function to estimate how close a given state is to the goal.

They make use of:

- Initial state
- Goal state
- Actions available between states
- Heuristic information

1.1 What is a Heuristic?

A heuristic function, usually denoted as h(n), estimates the cost from a node n to the goal.

Properties of a heuristic:

- It is problem-specific
- It provides guidance, not certainty
- A good heuristic improves efficiency

Examples:

- Straight-line distance in map navigation
- Number of misplaced tiles in 8-puzzle
- Manhattan distance in grid problems

1.1.1 Heuristic Function in Python

A heuristic is typically implemented as a dictionary or a function:

```
heuristic = {
    'A': 6,
    'B': 4,
    'C': 2,
    'D': 3,
    'E': 1,
    'G': 0
```

```
}
```

or

```
def h(state):
    return heuristic[state]
```

1.2 Types of Informed Search Algorithms

1.2.1 Greedy Best-First Search (GBFS)

Greedy Best-First Search expands the node that appears closest to the goal, based only on the heuristic value.

Evaluation function: f(n) = h(n)

GBFS ignores the path cost and focuses solely on the estimated distance to the goal.

- Create a priority queue ordered by h(n)
- Insert the initial node
- While the queue is not empty:
- Remove the node with the smallest h(n)
- Check if it is the goal
- Expand neighbors
- Add unexplored neighbors to the queue

1.2.2 A* Search Algorithm

A* is one of the most important and widely used search algorithms. It combines the strengths of Uniform Cost Search and Greedy Best-First Search.

Evaluation function: f(n) = g(n) + h(n)

Where:

- g(n): Cost from start node to node n
- h(n): Estimated cost from n to goal

A* selects the node with the lowest total estimated cost.

- Create a priority queue ordered by f(n)
- Insert the initial node with cost 0
- While the queue is not empty:

- Pop the node with lowest f(n)
- Check if it is the goal
- Expand neighbors
- Compute new g(n), h(n), and f(n)
- Update queue if a better path is found

In summary:

Algorithm	Evaluation Function	Optimal	Complete	Memory
GBFS	h(n)	No	No	Low
A*	g(n) + h(n)	Yes	Yes	High

2 Game Playing in Artificial Intelligence

Game playing is one of the classical problems in Artificial Intelligence. In game-playing problems, an intelligent agent must make decisions by considering possible future moves and the responses of an opponent.

These problems are usually modeled as:
- Two-player
- Turn-based
- Zero-sum games

Examples include:
- Tic-Tac-Toe
- Chess
- Checkers
- Connect Four

2.1 Characteristics of Game Playing Problems

- There are two players:
  - MAX (the AI or computer)
  - MIN (the opponent)
- Players take turns
- One player's gain is the other player's loss
- Both players are assumed to play optimally

2.2 Game State Representation

In game playing:

- A state represents a configuration of the game board
- The initial state is the starting position
- Terminal states are game-ending positions (win, lose, or draw)
- Each state has possible actions leading to new states

The game is represented as a game tree.

2.2.1 Game Tree

A game tree is a tree structure where:

- Each node represents a game state
- Each edge represents a possible move
- Levels of the tree alternate between MAX and MIN players
- MAX nodes: try to maximize the score
- MIN nodes: try to minimize the score

2.3 Utility (Evaluation) Function

At terminal states, a utility function assigns a numeric value indicating the outcome of the game. Typical utility values:

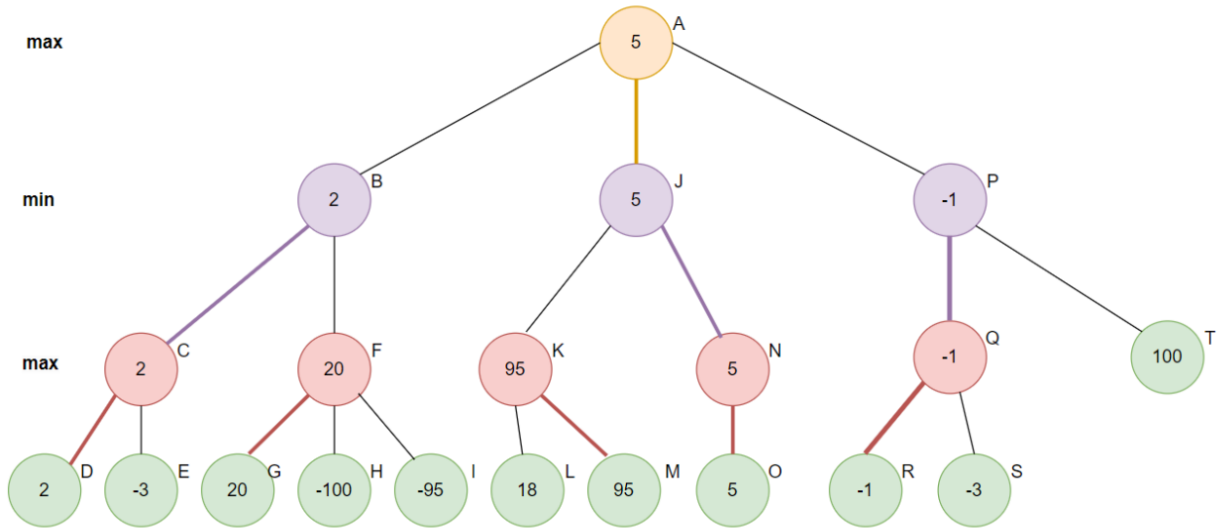
- Win for MAX: +1 (or positive value)
- `Loss for MAX: -1 (or negative value)`
- Draw: 0

3 Minimax Algorithm

The Minimax algorithm is a recursive decision-making algorithm used for optimal play in two-player games.

It assumes:

- Both players play optimally
- MAX tries to maximize the utility
- MIN tries to minimize the utility



Algorithm:

- Start from the root node
- If the node is a terminal state:
  - Return its utility value
- If it is MAX's turn:
  - Evaluate all child nodes
  - Return the maximum value
- If it is MIN's turn:
  - Evaluate all child nodes
  - Return the minimum value

```
MINIMAX(node, depth, isMaxPlayer):

  if node is terminal or depth == 0:
    return utility(node)

  if isMaxPlayer:
      bestValue = −∞
    for each child of node:
      value = MINIMAX(child, depth-1, False)
      bestValue = max(bestValue, value)
    return bestValue

  else:
```

```
    bestValue = +∞
    for each child of node:
        value = MINIMAX(child, depth-1, True)
        bestValue = min(bestValue, value)
    return bestValue
```

3.1 Node Representation

A node in a game tree may contain:

- Game state
- List of possible moves
- Player type (MAX or MIN)
- Utility value (for terminal nodes)

For example:

```
class Node:
    def __init__(self, value=None, children=None):
        self.value = value
        self.children = children
```